a2kit Installation Guide

This guide covers everything you need to install, what each piece is for, and how to confirm it works. If you just want the shortest path, the **minimum install** (Python plus two packages) is enough to convert files and preview them with no Apple II at all.

1. What you are installing

a2kit has two halves:

- **On your computer:** a set of Python command-line tools (`./a2kit …`) that convert and send files. This is what you install.
- **On the Apple II:** small 6502 programs. The finished ones are **already included** (`receiver.bin` , `sequencer.bin` , `video.bin`), along with **`a2kit-boot.dsk`** , a disk that boots straight into the receiver (and `a2kit-boot-19200.dsk` , the same at a sixth of the speed, as a fallback). Without the disk you type the receiver in once per power-up; the other programs are uploaded for you. You only need a 6502 assembler if you want to change them.

2. Requirements at a glance

Software on your computer

Item	Needed for	Required?	Tested with
Python 3.11 or newer	Everything	**Yes**	3.11, 3.12, 3.13
pillow 10.1+	Images, GIFs, previews	**Yes**	12.3, 13.0
numpy	Video and audio conversion	**Yes**	2.4, 2.5
pyserial	Sending to the Apple II	**Yes** (for sending)	3.5
py65	Running the tests	Only for `./a2kit test`	1.2
cc65 (`ca65` , `ld65`)	Rebuilding the 6502 programs	Only if you change them, or the slot or speed in `runtime/config.inc`	2.19
make	`./a2kit build` , `./a2kit test`	Only with cc65	GNU Make 4.3
ffmpeg	Video files such as `.mp4`	Optional (GIFs and frame folders don't need it)	6.1

Item	Needed for	Required?	Tested with
pandoc + **Chrome/Chromium**	Rebuilding the PDF docs	Optional	pandoc 3.1, Chrome 153

Tested setup

a2kit was first run on real hardware on 2 November 2026, on this setup:

Part	Tested with
Apple II	1979 Apple II+, 48K
Serial card	Super Serial Card II in slot 2, at 19200 and 115200 baud
Disk	Floppy Emu, booting the 19200 and 115200 disks
Computer	Dell Optiplex 790, Linux Mint, Python 3.12
Serial port	The computer's built-in serial port (`/dev/ttyS0` , a 16550A); from 8 November also a USB-to-RS-232 adapter with an FTDI FT232R chip (`/dev/ttyUSB0`)
Display	A colour TV on composite video, with its colour turned off

What worked: booting the disk straight into the receiver; `ping` ; a picture upload; the starter show with sound; the bouncing-ball video with and without sound. All the sounds (the jingle, the chirp and the boings between video frames) played from the built-in speaker. Measured from a recording of the jingle, the notes were within about 0.5% of their intended pitch, the once-every-64 ms page-change tick wasn't measurable, and the main artefact was a high whine at the 7,972 Hz sample rate. Two problems turned up and were fixed: a show that used hi-res page 2 ended on a screen of garbage (it now ends on text page 1), and the booted receiver's banners piled up session after session (it now clears the screen each time).

Measured on that setup at 19200 baud:

	Measured	Predicted
`ping` round trip (40-byte packet)	27.4 ms, of which 2.2 ms is the port	27.3 ms, 2.1 ms
Upload speed	1,719 bytes/s, 90% of the line rate	89–91%
An 8 KB picture	about 4.8 s	4.7–4.8 s
The starter show's upload	about 15 s	14.9 s
Bouncing ball with sound	about 7.6 s, no damage after any of the 15 sound bursts	7.7 s

Measured at 115200 baud on the same setup, booting the 115200 disk (3 November 2026), now `a2kit-boot.dsk` :

	Measured	Predicted
`ping` round trip (40-byte packet)	5.5 ms, of which 0.6 ms is the port	not predicted
Upload speed	about 8,730 bytes/s, 76% of the line rate	about 7,800 bytes/s, 68%
Upload speed with `--chunk 255` (8 November)	9,157 bytes/s, 79%	not predicted
An 8 KB picture	0.9 s	about 1.0 s
The starter show's upload	2.9 s	about 3.0 s
Bouncing ball with sound	about 7.7 s, 15 sound bursts	7.7 s
Stress clip: 52 frames at the full 1,824 bytes/frame (95% of the line), no sound	about 15 s, test card clean afterwards	15.0 s
The same clip with the jingle, in 27 sound bursts	about 18 s, 27 bursts on schedule, test card clean afterwards	18.1 s

The port's delay per packet was smaller at 115200 (0.6 ms) than at 19200 (2.2 ms), which is why uploads beat the prediction. The stress clip is `examples/stress` (Quick Start): eight large moving rings and a spinning cross, ending on a still grid that would show any leftover damage.

USB adapter compared with the built-in port at 115200 baud (8 November 2026). The adapter is an FTDI FT232R; the rest of the setup is unchanged. Each figure is from `a2kit ping`, 20 pings and 64 speed-test packets, with every packet acknowledged:

Port	Port delay per packet	Upload, 128-byte packets	Upload, 255-byte packets
Built-in `/dev/ttyS0`	0.6 ms	8,743 bytes/s, 76%	9,157 bytes/s, 79%
FTDI adapter, latency timer at its default 16 ms	11.1 ms	8,011 bytes/s, 70%	7,979 bytes/s, 69%
FTDI adapter, latency timer at 1 ms	1.1 ms	8,546 bytes/s, 74%	9,119 bytes/s, 79%

With the 16 ms timer the adapter passes received bytes on only when the timer runs out, so every packet takes a whole number of 16 ms periods. A 128-byte packet needs about 14 ms and takes one period (128 bytes / 16 ms = 8,000 bytes/s); a 255-byte packet needs about 27 ms and takes two (7,970 bytes/s), which is why larger packets didn't help. At 1 ms the adapter is within about 0.5 ms per packet of the built-in port. a2kit now sets the 1 ms timer itself whenever a command opens the port (see `a2kit ping` in the User Manual).

Everything else on both ports (8 November 2026). The same script ran each Quick Start step over the USB adapter, then over the built-in port. The USB adapter had just been unplugged and plugged back in, so its timer was at 16 ms until the first command set it to 1 ms. For the built-in run, the receiver was booted from the current `a2kit-boot.dsk` on the Floppy Emu, and the code in memory matched `receiver.bin`. On both ports every step ran with no retries, the receiver was listening again afterwards with no crash recorded, the pictures, sounds and videos all looked and sounded right, and the stress clip's final grid was

clean. Times run from starting the command to its end, so they include starting the tool and uploading the player and any sound:

	Built-in `/dev/ttyS0`	FTDI adapter
`ping` speed test, 128- and 255-byte packets	8,734 and 9,141 bytes/s	8,546 and 9,118 bytes/s
An 8 KB picture, `send out/logo.a2b`	1.0 s	1.1 s
The same with `--chunk 255`	1.0 s	1.0 s
The starter show, with `--install` (25,724 bytes)	3.3 s	3.4 s
Bouncing ball (a 6.0 s clip)	7.2 s	7.2 s
Bouncing ball with sound (7.7 s, 15 sound bursts)	9.3 s	9.3 s
Stress clip (15.0 s)	16.2 s	16.2 s
Stress clip with the jingle (18.1 s, 27 sound bursts)	20.1 s	20.1 s

With the timer at 1 ms, the adapter is as good as the built-in port in practice: uploads differ by about 0.1 s, and video, which sends without waiting for replies, not at all.

Not yet tried on real hardware: a USB-to-RS-232 adapter other than the FTDI one above; typing the receiver in instead of booting the disk; a real floppy; any other Apple II model; macOS or Windows; longer videos, busy videos at 19200, and `--scrub`; recovery from damaged bytes on a noisy line. If you try one of these, the Roadmap lists what to note down.

Hardware

Everything here is the hardware a2kit is **designed for**. One setup has been tested on real hardware so far (Tested setup); the rest follows from how the hardware works but hasn't been tried. You need no Apple II at all to convert files and look at previews.

Your computer

- Any computer that runs Python 3.11 or newer (see *Operating systems* below).
- An **RS-232 serial port**, either of:
  - a **built-in serial port** (`/dev/ttyS0` on Linux, `COM1` on Windows), often a DB-9 on the back of older desktops. This is the tested setup, and it adds the least delay per packet, though a tested FTDI adapter came within about 0.5 ms once its latency timer was turned down (see `a2kit ping` in the User Manual);
  - a **USB-to-RS-232 adapter** (FTDI, Prolific, CH340, CP210x and similar chips are all fine). It must be a true **RS-232** adapter, normally with a **DB-9** plug. A bare "USB-TTL" or "USB-UART" board (3.3 V or 5 V logic, usually on pin headers) will **not** work with a Super Serial Card, and connecting one could damage it.

The Apple II

Model	Status	Notes
Apple II+	**Tested** (setup)	With a Super Serial Card in slot 2.
Apple II (original), IIe	**Designed for**	As the II+. The original Apple II boots the disk with `6` then Ctrl-P at the Monitor, or `PR#6` from BASIC.
Apple IIc	Might work (untested)	Its built-in modem port uses the same 6551 chip at the same addresses as a card in slot 2, so a2kit may work unchanged. You need a IIc serial cable, since the port isn't a DB-25.
Apple IIc Plus	Might work (untested)	As the IIc, but it runs faster than 1 MHz unless set to normal speed (see *Speed*).
Apple IIgs	Only with a real Super Serial Card (untested)	The IIgs's own serial ports use a different chip (a Zilog SCC) that a2kit doesn't drive. Put a Super Serial Card in slot 2, set slot 2 to "Your Card" and the system speed to "Normal" in the Control Panel.
Clones and others	Unknown	Anything with a Disk II-compatible boot, a 6551 at the slot-2 addresses and a 1.02 MHz 6502 or 65C02 should behave like a II+.

- **Memory: 48K.** Shows and video with sound keep their sound in memory up to `$BFFF`.
- **Speed: the normal 1.02 MHz.** The sound and the pauses in a show are timed by counting CPU cycles. On a faster machine (an accelerator card, a IIgs in Fast mode, a IIc Plus at full speed), sound plays at the wrong pitch and pauses are too short, so switch the accelerator off or set normal speed. Both the original 6502 and the 65C02 (enhanced IIe, IIc) are fine: the timed code takes the same number of cycles on each.
- **Serial card: an Apple Super Serial Card**, or a compatible card built on the 6551 chip, in **slot 2**. The slot can be changed in `runtime/config.inc` (then run `./a2kit build`). The card's baud DIP switches don't matter: a2kit sets the speed itself (115200 baud, or 19200 with `a2kit-boot-19200.dsk`; 8 data bits, no parity, 1 stop bit). 115200 runs the card's 6551 straight from its crystal, as ADTPro does, and needs no modification; a short, sound cable matters more at that speed. Some cards have been refitted with a modern WDC 65C51 chip, which always reports its transmitter as ready. a2kit only ever sends one reply byte at a time, long after the previous one has gone, so it shouldn't be affected, but this is untested.
- **Display:** any. A monochrome or green-screen monitor shows the pictures best; colour monitors and TVs add coloured fringes to hi-res graphics.
- **Sound:** the built-in speaker. No sound card is needed (a Mockingboard isn't used).
- **Not needed:** a language card, an 80-column card, or any DOS. (DOS would take memory that sound needs.)

The cable

- The Super Serial Card's cable ends in a **DB-25** connector; most USB adapters have a **DB-9**. You'll usually need a DB-9-to-DB-25 cable or adapter.
- Whether transmit and receive need to be crossed (a *null-modem* adapter) depends on the card's jumper block. Some setups also need the card's handshake inputs (CTS, DCD, DSR) held active. Check the card's manual; Connect the Apple II has more.

To use the boot disk (optional; you can type the receiver in instead)

- A **16-sector Disk II controller** (standard from DOS 3.3 onwards; the earlier 13-sector controllers can't boot it) with a drive, or a **Floppy Emu** or similar drive emulator, or an emulator on your computer. See The boot disk.

Operating systems: developed and tested on **Linux**. The Python tools use nothing platform-specific, so macOS and Windows *should* work, but they have **not been tested** there.

Pick your level

You want to…	Install
Convert pictures, sound and video and look at previews	Python + `pillow` + `numpy`
…and send them to a real Apple II	+ `pyserial` and the hardware
…and play `.mp4` files	+ `ffmpeg`
Change the 6502 programs, or run the tests	+ `cc65`, `make`, `py65`

3. Install, step by step

Step 1. Unpack

```
tar xzf a2kit-starter.tar.gz
cd a2kit
```

Step 2. Python 3.11 or newer

Check what you have:

```
python3 --version
```

If it's older than 3.11, or missing, install a current Python:

System	Command or download
Debian / Ubuntu	`sudo apt install python3 python3-venv python3-pip`
macOS	`brew install python` (Homebrew), or the installer from python.org
Windows	The installer from python.org. **Tick "Add python.exe to PATH"** on the first screen.

On Windows the command is usually `python` or `py -3` instead of `python3`.

Step 3. Create a virtual environment and install the packages

A *virtual environment* keeps a2kit's packages separate from the rest of your system. Recent Linux and Homebrew Pythons require it (without one, `pip` stops with an `externally-managed-environment` error).

Linux and macOS:

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
```

Windows (PowerShell):

```
py -3 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

Activate the environment (`source .venv/bin/activate`) in each new terminal before using a2kit. If you also want to run the tests, install `requirements-dev.txt` instead, which adds `py65` (and `pypdf`, used only to rebuild these PDFs).

> **Careful:** the package is called **`pyserial`**. A different package called `serial` exists and will *not* work. If you see `No module named 'serial'` after installing something, run `python3 -m pip uninstall serial` and install `pyserial`.

Step 4. Optional tools

cc65 and make (rebuild the 6502 programs, run the tests):

System	Command
Debian / Ubuntu	`sudo apt install cc65 make`
macOS	`brew install cc65` (make comes with the Xcode command-line tools: `xcode-select --install`)
Windows	Easiest is **WSL** (Windows Subsystem for Linux): install Ubuntu there and follow the Linux steps. Native Windows has no `make`, but you don't need it unless you're rebuilding: the included `.bin` files work as they are.

ffmpeg (video files): `sudo apt install ffmpeg`, `brew install ffmpeg`, or the downloads and package-manager options at ffmpeg.org for Windows.

Step 5. Check the install

```
./a2kit doctor
```

You should see `ok` against Python, `pillow`, `numpy`, `pyserial`, and the three `.bin` files, ending with `all required items present`. Anything the command marks `MISSING` is required; anything marked `optional` can wait until you need it.

On Windows, where the `./a2kit` shell launcher doesn't run, use `python tools\a2kit.py doctor` (and the same in place of `./a2kit` everywhere else).

Then confirm the conversions work with no hardware at all:

```
mkdir -p out
./a2kit image examples/logo.png -o out/logo.a2b --preview out/logo_preview.png
```

Open `out/logo_preview.png`. If you see the title card as black-and-white dots, the install is working. To run everything, `./a2kit test` (needs the cc65, make and `py65` items; takes under a minute).

4. Connect the Apple II

This step is only needed to send to real hardware.

Find the serial port

With the adapter plugged in, list the ports pyserial can see (works on every system):

```
python3 -m serial.tools.list_ports
```

System	What the port looks like	Notes
Linux	Built-in: `/dev/ttyS0` (or `ttyS1`). USB: `/dev/ttyUSB0` or `/dev/ttyACM0`	See "permission denied" below. `sudo dmesg | grep ttyS` lists built-in ports.
macOS	`/dev/cu.usbserial-…`	Use the **`cu.`** name, not `tty.`. USB-serial chips (FTDI, CH340, CP210x) may need the vendor's driver on older macOS versions.
Windows	Built-in: usually `COM1`. USB: `COM3` or another number	Also visible in Device Manager under *Ports (COM & LPT)*. Install the adapter's driver if it shows an error.

Permission to use the port (Linux)

If a2kit reports `Permission denied` for `/dev/ttyUSB0`, add yourself to the group that owns serial ports, then **log out and back in**:

```
sudo usermod -aG dialout $USER      # Debian/Ubuntu; the group is "uucp" on Arch
```

Cabling

The Super Serial Card needs a cable or adapter set (DB-25 on the card's side, usually DB-9 on the adapter's) that connects its transmit, receive and ground lines to your USB-to-RS-232 adapter, normally with transmit and receive crossed (a *null-modem* arrangement). The right wiring depends on your card's jumper-block position and your adapter, so check your card's manual. Settings on the computer side are **115200 baud (or 19200), 8 data bits, no parity, 1 stop bit**, and a2kit sets them for you. Built-in 16550 ports and common USB adapters all handle 115200.

Depending on the card's jumpers and your cable, the card's handshake inputs (CTS, DCD, DSR) may also need to be held active before it will send or receive. If the receiver banner is showing on the Apple II but uploads fail with `no ACK`, suspect wiring, jumpers and handshake lines first.

The Apple II side, booting the disk or typing in the receiver, is in the Quick Start.

Test the link

With the receiver running on the Apple II, check the whole path (port, cable, card and receiver) in one go:

```
./a2kit ping --port /dev/ttyUSB0
```

It should end with `Link OK`, and the Apple II's bottom line should read `A2KIT PING 10 OF 10`. Otherwise its verdict says whether nothing came back, the speed doesn't match, or packets arrive damaged. It also measures how much delay your port adds to each packet; the User Manual explains how to reduce it.

5. The boot disk

`a2kit-boot.dsk` boots an Apple II straight into the receiver, so you don't type it in. `a2kit-boot-19200.dsk` is the same at 19200 baud (see Line speed); everything below applies to both. It is a standard 140 KB 5.25-inch image (35 tracks, 16 sectors, DOS 3.3 order). Any of these will do:

Way	What to do
Floppy Emu or a similar SD-card drive emulator	Copy `a2kit-boot.dsk` to its card and select it as a 5.25-inch disk.

Way	What to do
A real floppy	Write the image with a disk-image tool for your setup, for example ADTPro (over the same serial cable) or a flux-level writer such as Applesauce. Follow that tool's instructions for writing a `.dsk` image.
An emulator	Open the image as the disk in drive 1. A good way to check the image before you write a real floppy.

Requirements on the Apple II side: a 16-sector Disk II controller with the disk in drive 1 (see Hardware).

`./a2kit disk` remakes the image; you only need it after rebuilding the receiver (see the User Manual).

6. Problems with the install

Symptom	Fix
`error: externally-managed-environment`	Use a virtual environment (Step 3).
`python3: command not found` (Windows)	Use `python` or `py -3`; reinstall Python with *Add to PATH* ticked.
`./a2kit: Permission denied`	`chmod +x a2kit`, or run `python3 tools/a2kit.py …` instead.
`./a2kit: command not found` / won't run (Windows)	Use `python tools\a2kit.py …`.
`ModuleNotFoundError: No module named 'PIL'` (or `numpy`)	The environment isn't active, or packages weren't installed into it: activate it and re-run Step 3.
`No module named 'serial'`	Install **`pyserial`**, not `serial` (see the note in Step 3).
`ModuleNotFoundError: No module named 'tomllib'`	Python is older than 3.11. Install a newer one (Step 2).
`ca65: command not found` when building	Install cc65 (Step 4). You don't need it unless you're rebuilding.
`make: command not found` (Windows)	Use WSL, or skip rebuilding; the included `.bin` files work.
`ffmpeg isn't on PATH`	Install ffmpeg (Step 4), or convert your video to a GIF or a folder of frames.
`Permission denied: '/dev/ttyUSB0'`	See *Permission to use the port*.
Port is busy, or disappears a moment after plugging in (Linux)	Another program has it open. A background service such as ModemManager can grab new serial devices; close other terminal programs, or stop that service while you use a2kit.

Symptom	Fix
`doctor` says a `.bin` file is missing	Restore it from the original bundle, or run `./a2kit build` (needs cc65).

7. Upgrading and removing

- **Upgrade:** unpack a newer bundle into a new folder, then re-run Step 3 there. Your own show scripts and assets are plain files you can copy across.
- **Remove:** delete the `a2kit` folder (the virtual environment lives inside it as `.venv`). Nothing is installed system-wide by the steps above except the optional tools in Step 4.

Where next

- Quick Start: your first picture on the screen.
- User Manual: every command in detail.